

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Системы компьютерной обработки изображений

Лабораторная работа № 3

«Расчет интеграла функции на заданном промежутке методом Монте-Карло»

Выполнил
студент

Стрельбицкий Илья Павлович

Группа № Р3413

Преподаватель: Жданов Дмитрий Дмитриевич

г. Санкт-Петербург 2026

Цель

Изучить базовые методы Монте-Карло, используемые для интегрирования функций на заданном интервале, и оценить точность интегрирования и основные преимущества и недостатки этих методов.

Задание

Задана функция $f(x) = x^2$. Вычислить интеграл данной функции на интервале $[2, 5]$. Интеграл вычисляется:

1. Аналитически.
2. Простым интегрированием методом Монте-Карло.
3. Интегрированием методом Монте-Карло со стратификацией. Использовать два варианта разбиения: с шагом 1 и с шагом 0.5.
4. Интегрированием методом Монте-Карло с выборкой по значимости. Использовать три варианта плотности вероятности $p_1(x) = x, p_2(x) = x^2, p_3(x) = x^3$.
5. Интегрированием методом Монте-Карло с многократной выборкой по значимости. Использовать две функции плотности вероятности $p(x) = x, p(x) = x^3$. В качестве весов использовать среднюю плотность вероятности (первый вариант расчета $w_1(x) = \frac{p_1(x)}{p_1(x)+p_2(x)}, w_2(x) = \frac{p_2(x)}{p_1(x)+p_2(x)}$) и средний квадрат двух плотностей вероятности (второй вариант расчета $w_1(x) = \frac{p_1^2(x)}{p_1^2(x)+p_2^2(x)}, w_2(x) = \frac{p_2^2(x)}{p_1^2(x)+p_2^2(x)}$).
6. Интегрированием методом Монте-Карло с использованием русской рулетки. Использовать три точки отсечки: $R_1 = 0.5, R_2 = 0.75, R_3 = 0.95$.

При вычислении использовать программный генератор псевдослучайных чисел. Использовать разное число выборок при расчете интеграла: $N_1 = 100, N_2 = 1000, N_3 = 10000, N_5 = 100000$.

Результаты представить в виде отчета, в который содержит таблицы значений истинного интеграла, интеграла, вычисленного каждым из методов Монте-Карло, погрешности вычислений и оценки погрешности $\Delta I = \frac{I_{true}}{N}$.

Представить код программы. Язык программирования произвольный.

Ход выполнения

1) Аналитическое вычисление интеграла:

Функция: $f(x) = x^2$

Интервал: $[2, 5]$

$$I_{true} = \int_2^5 x^2 dx = \left. \frac{x^3}{3} \right|_2^5 = \frac{5^3}{3} - \frac{2^3}{3} = \frac{125}{3} - \frac{8}{3} = \frac{117}{3} = 39$$

2) Описание и реализация методов Монте-Карло

Для генерации числовых выборок используется генератор случайных чисел библиотеки `numpy`.

Для воспроизводимости результатов задается фиксированное зерно генератора: `np.random.seed(10)`

2.1) Простое интегрирование методом Монте-Карло

Идея метода:

Случайно выбираем точки x_i равномерно на $[a, b]$, и оцениваем интеграл как среднее значение функции, умноженное на длину отрезка:

$$x_i \sim U(a, b), i = 1, \dots, N,$$

$$I \approx (b - a) \cdot \frac{1}{N} \sum_{i=1}^N f(x_i).$$

2.2) Метод Монте-Карло со стратификацией

Идея метода:

Интервал $[a, b]$ разбивается на несколько подинтервалов (страт), и в каждом подинтервале независимым образом берутся выборки. Это уменьшает дисперсию оценки.

Пусть интервал разделён на отрезки: $[a, b] = \bigcup_{k=1}^K [x_k, x_{k+1}]$,

где шаг разбиения:

- вариант 1: $\Delta x = 1$
- вариант 2: $\Delta x = 0.5$

На каждом подотрезке $[x_k, x_{k+1}]$ выбираем N_k точек: $x_i^{(k)} \sim U(x_k, x_{k+1})$

Оценка локального интеграла: $\hat{I}_k = (x_{k+1} - x_k) \cdot \frac{1}{N_k} \sum_{i=1}^{N_k} f(x_i^{(k)})$

Общая оценка: $\hat{I} = \sum_{k=1}^K \hat{I}_k$

2.3) Интегрирование методом Монте-Карло с выборкой по значимости

Идея метода:

Выбирать точки не равномерно, а по распределению, близкому к форме подынтегральной функции, чтобы уменьшить дисперсию.

Пусть $p(x)$ — плотность вероятности на $[a, b]$, тогда:

$$I = \int_a^b f(x) dx = \int_a^b \frac{f(x)}{p(x)} p(x) dx \approx \hat{I} = \frac{1}{N} \sum_{i=1}^N \frac{f(x_i)}{p(x_i)},$$

где $x_i \sim p(x)$.

В работе используются плотности вида: $p_k(x) \propto x^k$, $k = 1, 2, 3$.

Нормированная плотность на $[a, b]$: $p_k(x) = \frac{(k+1)x^k}{b^{k+1} - a^{k+1}}$, $a \leq x \leq b$

Инверсия распределения:

$$\text{Функция распределения: } F_k(x) = \int_a^x p_k(t) dt = \frac{x^{k+1} - a^{k+1}}{b^{k+1} - a^{k+1}}$$

$$\text{Если } u \sim U(0.1), \text{ то: } u = F_k(x) \Rightarrow x = (u(b^{k+1} - a^{k+1}) + a^{k+1})^{\frac{1}{k+1}}$$

Особый случай:

При $k = 2$, $p(x) \propto x^2$ совпадает по форме с $f(x) = x^2$, поэтому величина $\frac{f(x)}{p(x)}$ становится константой, и дисперсия оценки равна нулю. Это видно в результатах: метод с $p(x) \propto x^2$ всегда даёт 39.

2.4) Многократная выборка по значимости – Multiple Importance Sampling, MIS

Используются две плотности: $p_1(x) \propto x$, $p_2(x) \propto x^3$

Идея метода:

Комбинировать выборки из нескольких плотностей и взвешивать вклады так, чтобы уменьшить дисперсию и избежать доминирования плохих плотностей.

Общая схема:

- берем N_1 выборок из p_1 : $x_i^{(1)}$
- берем N_2 выборок из p_2 : $x_j^{(2)}$
- для каждого значения считаем веса $w_1(x)$, $w_2(x)$
- оценка интеграла: $\hat{I} = \frac{1}{N_1} \sum_{i=1}^{N_1} \frac{w_1(x_i^{(1)})f(x_i^{(1)})}{p_1(x_i^{(1)})} + \frac{1}{N_2} \sum_{j=1}^{N_2} \frac{w_2(x_j^{(2)})f(x_j^{(2)})}{p_2(x_j^{(2)})}$

Реализованы два варианта весов:

Средняя плотность (balance heuristic):

$$w_1(x) = \frac{p_1(x)}{p_1(x) + p_2(x)}, w_2(x) = \frac{p_2(x)}{p_1(x) + p_2(x)}.$$

Средний квадрат плотностей (power heuristic):

$$w_1(x) = \frac{p_1^2(x)}{p_1^2(x) + p_2^2(x)}, w_2(x) = \frac{p_2^2(x)}{p_1^2(x) + p_2^2(x)}$$

2.5) Метод русской рулетки

Метод русской рулетки используется для случайного усечения вычислений без смещения оценки.

В данной задаче реализовано следующее:

- с вероятностью R «выживаем» и считаем вклад,
- с вероятностью $1 - R$ вклад равен нулю.

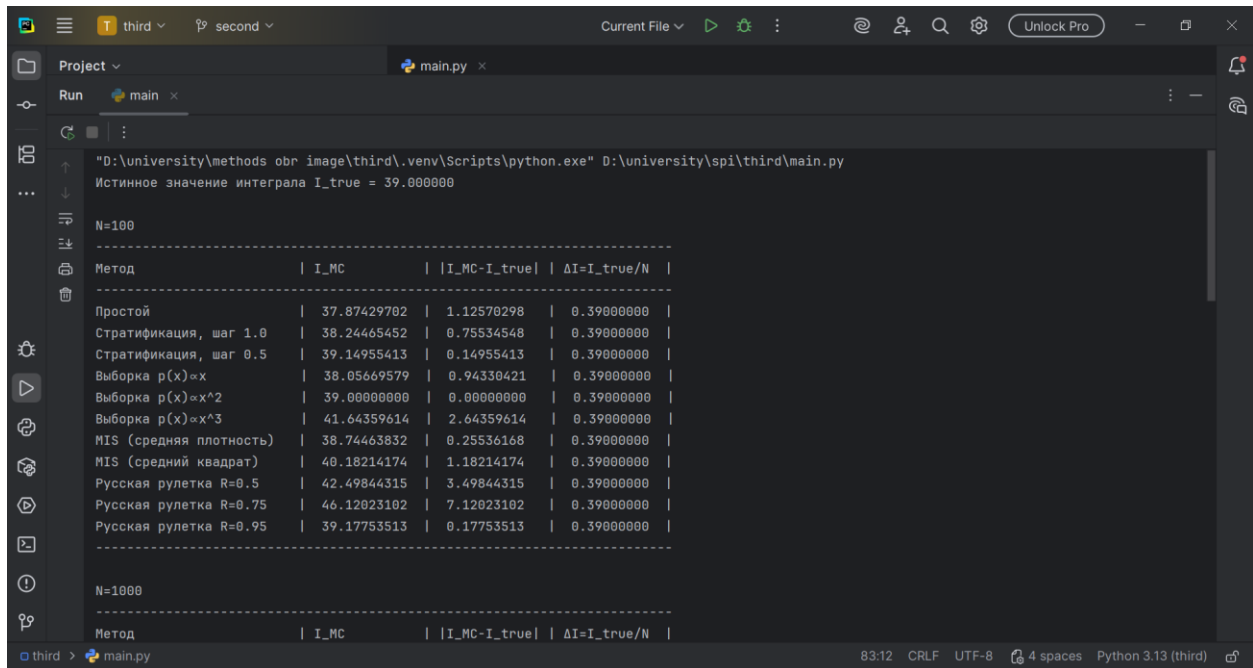
Если элемент выжил, генерируем $x \sim U(a, b)$ и добавляем вклад: $\text{sample} = \frac{(b-a)f(x)}{R}$

Математическое ожидание:

$$E[\text{sample}] = R \cdot \frac{(b-a)E[f(x)]}{R} + (1-R) \cdot 0 = (b-a)E[f(x)] = I,$$

то есть оценка несмещённая.

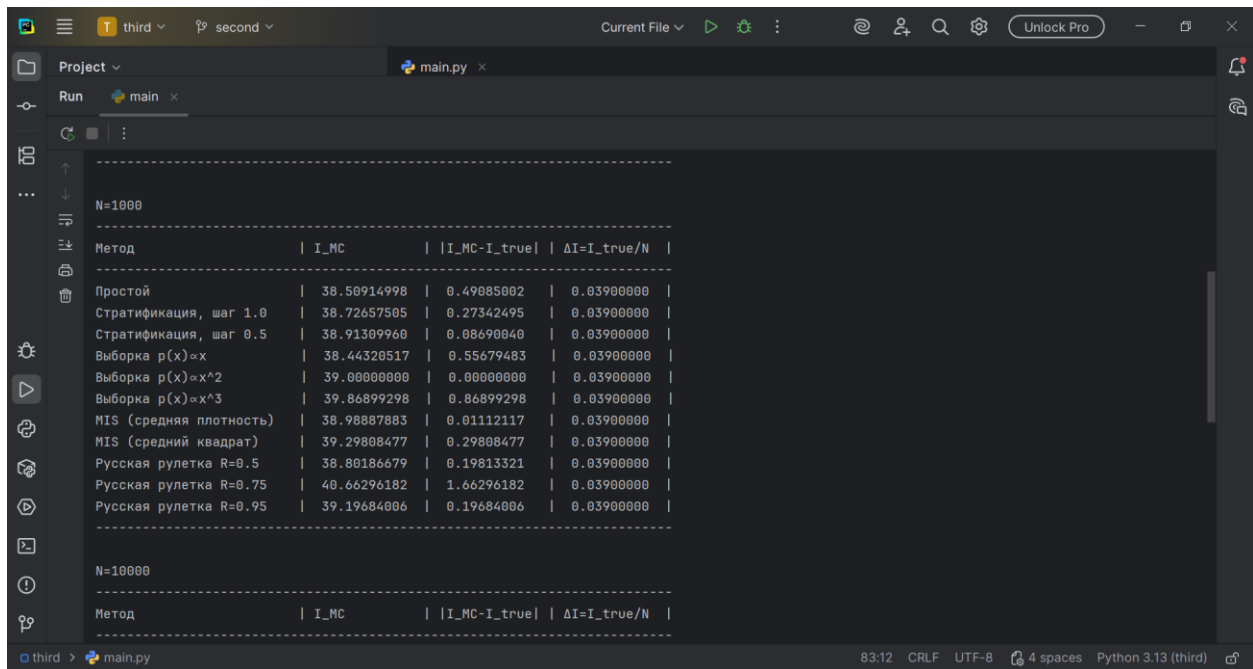
3) Результаты запуска программы:



```
"D:\University\methods obr image\third\.venv\Scripts\python.exe" D:\University\spi\third\main.py
Истинное значение интеграла I_true = 39.000000

N=100
-----
Метод          | I_MC          | |I_MC-I_true| | ΔI=I_true/N |
-----
Простой         | 37.87429702   | 1.12570298   | 0.39000000   |
Стратификация, шаг 1.0 | 38.24465452   | 0.75534548   | 0.39000000   |
Стратификация, шаг 0.5 | 39.14955413   | 0.14955413   | 0.39000000   |
Выборка p(x)~x   | 38.05669579   | 0.94330421   | 0.39000000   |
Выборка p(x)~x^2 | 39.00000000   | 0.00000000   | 0.39000000   |
Выборка p(x)~x^3 | 41.64359614   | 2.64359614   | 0.39000000   |
MIS (средняя плотность) | 38.74463832   | 0.25536168   | 0.39000000   |
MIS (средний квадрат) | 40.18214174   | 1.18214174   | 0.39000000   |
Русская рулетка R=0.5 | 42.49844315   | 3.49844315   | 0.39000000   |
Русская рулетка R=0.75 | 46.12023102   | 7.12023102   | 0.39000000   |
Русская рулетка R=0.95 | 39.17753513   | 0.17753513   | 0.39000000   |
-----

N=1000
-----
Метод          | I_MC          | |I_MC-I_true| | ΔI=I_true/N |
-----
```



```
N=1000
-----
Метод          | I_MC          | |I_MC-I_true| | ΔI=I_true/N |
-----
Простой         | 38.50914998   | 0.49085002   | 0.03900000   |
Стратификация, шаг 1.0 | 38.72657505   | 0.27342495   | 0.03900000   |
Стратификация, шаг 0.5 | 38.91309960   | 0.08690040   | 0.03900000   |
Выборка p(x)~x   | 38.44320517   | 0.55679483   | 0.03900000   |
Выборка p(x)~x^2 | 39.00000000   | 0.00000000   | 0.03900000   |
Выборка p(x)~x^3 | 39.86899298   | 0.86899298   | 0.03900000   |
MIS (средняя плотность) | 38.98887883   | 0.01112117   | 0.03900000   |
MIS (средний квадрат) | 39.29808477   | 0.29808477   | 0.03900000   |
Русская рулетка R=0.5 | 38.80186679   | 0.19813321   | 0.03900000   |
Русская рулетка R=0.75 | 40.66296182   | 1.66296182   | 0.03900000   |
Русская рулетка R=0.95 | 39.19684006   | 0.19684006   | 0.03900000   |
-----

N=10000
-----
Метод          | I_MC          | |I_MC-I_true| | ΔI=I_true/N |
-----
```

```
Project Alt+1 main.py x
Run main x
N=10000
-----
Метод | I_MC | | I_MC-I_true | | ΔI=I_true/N |
-----
Простой | 39.05937082 | | 0.05937082 | | 0.00390000 |
Стратификация, шаг 1.0 | 39.02166362 | | 0.02166362 | | 0.00390000 |
Стратификация, шаг 0.5 | 39.00539199 | | 0.00539199 | | 0.00390000 |
Выборка p(x) ∝ x | 38.91744360 | | 0.08255640 | | 0.00390000 |
Выборка p(x) ∝ x^2 | 39.00000000 | | 0.00000000 | | 0.00390000 |
Выборка p(x) ∝ x^3 | 38.93264374 | | 0.06735626 | | 0.00390000 |
MIS (средняя плотность) | 38.98718931 | | 0.01281069 | | 0.00390000 |
MIS (средний квадрат) | 38.98790776 | | 0.01209224 | | 0.00390000 |
Русская рулетка R=0.5 | 38.64257751 | | 0.35742249 | | 0.00390000 |
Русская рулетка R=0.75 | 38.66808671 | | 0.33191329 | | 0.00390000 |
Русская рулетка R=0.95 | 39.16116757 | | 0.16116757 | | 0.00390000 |
-----
N=100000
-----
Метод | I_MC | | I_MC-I_true | | ΔI=I_true/N |
-----
Простой | 39.04905085 | | 0.04905085 | | 0.00039000 |
```

```
Project main.py x
Run main x
Русская рулетка R=0.95 | 39.16116757 | | 0.16116757 | | 0.00390000 |
-----
N=100000
-----
Метод | I_MC | | I_MC-I_true | | ΔI=I_true/N |
-----
Простой | 39.04905085 | | 0.04905085 | | 0.00039000 |
Стратификация, шаг 1.0 | 39.01639009 | | 0.01639009 | | 0.00039000 |
Стратификация, шаг 0.5 | 39.00206329 | | 0.00206329 | | 0.00039000 |
Выборка p(x) ∝ x | 39.07280313 | | 0.07280313 | | 0.00039000 |
Выборка p(x) ∝ x^2 | 39.00000000 | | 0.00000000 | | 0.00039000 |
Выборка p(x) ∝ x^3 | 38.96158889 | | 0.03841111 | | 0.00039000 |
MIS (средняя плотность) | 38.99950900 | | 0.00049100 | | 0.00039000 |
MIS (средний квадрат) | 39.00426871 | | 0.00426871 | | 0.00039000 |
Русская рулетка R=0.5 | 38.87842680 | | 0.12157320 | | 0.00039000 |
Русская рулетка R=0.75 | 38.99702163 | | 0.00297837 | | 0.00039000 |
Русская рулетка R=0.95 | 39.03001349 | | 0.03001349 | | 0.00039000 |
-----
Process finished with exit code 0
```


Таблицы результатов:

$$I_{true} = 39$$

N=100

Метод	I	$I - I_{true}$	$\Delta I = \frac{I_{true}}{N}$
Простой	37.87429702	1.12570298	0.39000000
Стратификация, шаг 1.0	38.24465452	0.75534548	0.39000000
Стратификация, шаг 0.5	39.14955413	0.14955413	0.39000000
Выборка $p(x) \propto x$	38.05669579	0.94330421	0.39000000
Выборка $p(x) \propto x^2$	39.00000000	0.00000000	0.39000000
Выборка $p(x) \propto x^3$	41.64359614	2.64359614	0.39000000
MIS (средняя плотность)	38.74463832	0.25536168	0.39000000
MIS (средний квадрат)	40.18214174	1.18214174	0.39000000
Русская рулетка R=0.5	42.49844315	3.49844315	0.39000000
Русская рулетка R=0.75	46.12023102	7.12023102	0.39000000
Русская рулетка R=0.95	39.17753513	0.17753513	0.39000000

N=1000

Метод	I	$I - I_{true}$	$\Delta I = \frac{I_{true}}{N}$
Простой	38.50914998	0.49085002	0.03900000
Стратификация, шаг 1.0	38.72657505	0.27342495	0.03900000
Стратификация, шаг 0.5	38.91309960	0.08690040	0.03900000
Выборка $p(x) \propto x$	38.44320517	0.55679483	0.03900000
Выборка $p(x) \propto x^2$	39.00000000	0.00000000	0.03900000
Выборка $p(x) \propto x^3$	39.86899298	0.86899298	0.03900000
MIS (средняя плотность)	38.98887883	0.01112117	0.03900000
MIS (средний квадрат)	39.29808477	0.29808477	0.03900000
Русская рулетка R=0.5	38.80186679	0.19813321	0.03900000
Русская рулетка R=0.75	40.66296182	1.66296182	0.03900000
Русская рулетка R=0.95	39.19684006	0.19684006	0.03900000

N=10000

Метод	I	$I - I_{true}$	$\Delta I = \frac{I_{true}}{N}$
Простой	39.05937082	0.05937082	0.00390000
Стратификация, шаг 1.0	39.02166362	0.02166362	0.00390000
Стратификация, шаг 0.5	39.00539199	0.00539199	0.00390000
Выборка $p(x) \propto x$	38.91744360	0.08255640	0.00390000
Выборка $p(x) \propto x^2$	39.00000000	0.00000000	0.00390000
Выборка $p(x) \propto x^3$	38.93264374	0.06735626	0.00390000
MIS (средняя плотность)	38.98718931	0.01281069	0.00390000
MIS (средний квадрат)	38.98790776	0.01209224	0.00390000
Русская рулетка R=0.5	38.64257751	0.35742249	0.00390000
Русская рулетка R=0.75	38.66808671	0.33191329	0.00390000
Русская рулетка R=0.95	39.16116757	0.16116757	0.00390000

N=100000

Метод	I	$I - I_{true}$	$\Delta I = \frac{I_{true}}{N}$
Простой	39.04905085	0.04905085	0.00039000
Стратификация, шаг 1.0	39.01639009	0.01639009	0.00039000
Стратификация, шаг 0.5	39.00206329	0.00206329	0.00039000
Выборка $p(x) \propto x$	39.07280313	0.07280313	0.00039000
Выборка $p(x) \propto x^2$	39.00000000	0.00000000	0.00039000
Выборка $p(x) \propto x^3$	38.96158889	0.03841111	0.00039000
MIS (средняя плотность)	38.99950900	0.00049100	0.00039000
MIS (средний квадрат)	39.00426871	0.00426871	0.00039000
Русская рулетка R=0.5	38.87842680	0.12157320	0.00039000
Русская рулетка R=0.75	38.99702163	0.00297837	0.00039000
Русская рулетка R=0.95	39.03001349	0.03001349	0.00039000

4) Код программы на Python:

```
import numpy as np
from numpy import floating
from typing import Any

def f(x) -> floating[Any]:
    """Целевая функция"""
    return x ** 2

def analytical_integral() -> float:
    """Аналитическое вычисление интеграла"""
    return 39.0

class MonteCarlo:
    def __init__(self, a=2, b=5):
        self.a = a
        self.b = b
        self.true_value = analytical_integral()
        self.n_samples_list = [100, 1000, 10000, 100000]

    def mc_simple_uniform(self, N: int) -> floating[Any]:
        """
        Простое интегрирование методом Монте-Карло:
         $X \sim U(a, b)$ 
         $I \approx (b-a)/N * \sum f(X_i)$ 
        """
        x = np.random.uniform(self.a, self.b, size=N)
        return (self.b - self.a) * np.mean(f(x))

    def mc_stratified(self, N: int, step: float) -> float:
        """
        Стратификация по отрезкам длины step:
        [a, a+step], [a+step, a+2*step], ..., [..., b]
        В каждом страта-отрезке берём примерно одинаковое число выборок.
        """
        edges = np.arange(self.a, self.b + 1e-12, step)
        if abs(edges[-1] - self.b) > 1e-9:
            edges = np.append(edges, self.b)
        total_estimate = 0.0
        n_strata = len(edges) - 1
        # равномерно распределяем N по стратам
        base = N // n_strata
        remainder = N - base * n_strata
        for i in range(n_strata):
            n_i = base + (1 if i < remainder else 0)
            left, right = edges[i], edges[i + 1]
            x = np.random.uniform(left, right, size=n_i)
            # локальный интеграл
            total_estimate += (right - left) * np.mean(f(x))
        return total_estimate

    """
    =====
    Вспомогательные функции для выборки по значимости
     $p(x) \sim x^k$ ,
    """
```

```

k = 1, 2, 3 на [a, b]
=====
"""
def sample_power(self, N: int, k: int) -> floating[Any]:
    """
    Сэмплирование из распределения с плотностью
     $p(x) \propto x^k$  на  $[a, b]$  через метод обратной функции.
    CDF:  $F(x) = (x^{k+1} - a^{k+1}) / (b^{k+1} - a^{k+1}) \Rightarrow$ 
     $x = (u * (b^{k+1} - a^{k+1}) + a^{k+1})^{1/(k+1)}$ 
    """
    u = np.random.rand(N)
    return (
        (u * (self.b ** (k + 1) - self.a ** (k + 1)) +
         self.a ** (k + 1)) ** (1.0 / (k + 1))
    )

def pdf_power(self, x: floating[Any], k: int) -> floating[Any]:
    """
    Нормированная плотность:
     $p_k(x) = (k+1) * x^k / (b^{k+1} - a^{k+1}), x \in [a, b]$ 
    """
    return (k + 1) * x ** k / (self.b ** (k + 1) - self.a ** (k + 1))

def mc_importance(self, N: int, k: int) -> floating[Any]:
    """
    Интегрирование методом Монте-Карло с выборкой по значимости
    для плотности  $p_k(x) \sim x^k$  (нормированной).
     $I \approx 1/N * \sum f(x_i) / p_k(x_i)$ , где  $x_i \sim p_k$ .
    """
    x = self.sample_power(N, k)
    p = self.pdf_power(x, k)
    return np.mean(f(x) / p)
"""
=====
"""

def mc_mis(self, N: int, use_power_heuristic=False) -> floating[Any]:
    """
    Multiple Importance Sampling (Многократная выборка по значимости)
     $p_1(x) \sim x$ ,
     $p_2(x) \sim x^3$ 
    Два варианта весов:
    1)  $w_1 = p_1/(p_1+p_2)$ ;  $w_2 = p_2/(p_1+p_2)$ 
    2)  $w_1 = p_1^2/(p_1^2+p_2^2)$ ;  $w_2 = p_2^2/(p_1^2+p_2^2)$ 
    - N/2 выборки из  $p_1(x) \sim x$ 
    - N/2 выборки из  $p_2(x) \sim x^3$ 
    Веса считаем по заданным формулам.
    Возвращаем оценку интеграла.
    """
    N1 = N // 2
    # на случай нечётного N, но у нас все N четные
    N2 = N - N1
    # выборка из  $p_1$  (k=1)
    x1 = self.sample_power(N1, 1)
    p1_x1 = self.pdf_power(x1, 1)
    p2_x1 = self.pdf_power(x1, 3)

```

```

# выборка из p2 (k=3)
x2 = self.sample_power(N2, 3)
p1_x2 = self.pdf_power(x2, 1)
p2_x2 = self.pdf_power(x2, 3)
if use_power_heuristic:
    # второй вариант: средний квадрат плотностей
    w1_x1 = p1_x1 ** 2 / (p1_x1 ** 2 + p2_x1 ** 2)
    w2_x2 = p2_x2 ** 2 / (p1_x2 ** 2 + p2_x2 ** 2)
else:
    # первый вариант: средняя плотность
    w1_x1 = p1_x1 / (p1_x1 + p2_x1)
    w2_x2 = p2_x2 / (p1_x2 + p2_x2)
# оценка из выборок p1 и p2
estimate_first = np.mean(f(x1) * w1_x1 / p1_x1)
estimate_second = np.mean(f(x2) * w2_x2 / p2_x2)
# суммируем обе части
return estimate_first + estimate_second

def mc_russian_roulette(self, N: int, R: float) -> floating[Any]:
    """
    Метод Монте-Карло с русской рулеткой.
    Пусть с вероятностью R "выживаем" и считаем вклад,
    с вероятностью (1-R) - обнуляем вклад.
    Если выжили:  $X \sim U(a, b)$ , вклад =  $(b-a) * f(X) / R$ 
    Тогда  $E[\text{вклад}] = \int f(x) dx$  - оценка интеграла.
     $I \approx 1/N * \sum \text{sample}_i$ 
    """
    u = np.random.rand(N)
    survive_mask: np.ndarray[bool] = (u < R)
    M = survive_mask.sum()
    samples = np.zeros(N)
    if M > 0:
        x = np.random.uniform(self.a, self.b, size=M)
        samples[survive_mask] = (self.b - self.a) * f(x) / R
    return samples.mean()

def print_table(results: list, N: int) -> None:
    print(f"N={N}")
    header = f"{'Метод':25s} | {'I_MC' + ' ' * 8} | {'|I_MC-I_true|'} | {'ΔI=I_true/N':>10s} |"
    print("-" * len(header))
    print(header)
    print("-" * len(header))
    for name, I_est, err, delta_I in results:
        print(f"{name:25s} | {I_est:3.8f} | {err:3.8f} | {delta_I:3.8f} |")
    print("-" * len(header))
    print()

def main() -> None:
    # фиксируем зерно для воспроизводимости
    np.random.seed(10)
    I_true = analytical_integral()
    monte_carlo = MonteCarlo()
    print(f"Истинное значение интеграла I_true = {I_true:.6f}\n")
    for n in monte_carlo.n_samples_list:

```

```

results = []
delta_I = I_true / n # оценка погрешности
I_estimate = monte_carlo.mc_simple_uniform(n)
results.append(("Простой", I_estimate, abs(I_estimate - I_true),
delta_I))
I_estimate = monte_carlo.mc_stratified(n, step=1.0)
results.append(("Стратификация, шаг 1.0", I_estimate, abs(I_estimate
- I_true), delta_I))
I_estimate = monte_carlo.mc_stratified(n, step=0.5)
results.append(("Стратификация, шаг 0.5", I_estimate, abs(I_estimate
- I_true), delta_I))
I_estimate = monte_carlo.mc_importance(n, k=1)
results.append(("Выборка  $p(x) \propto x$ ", I_estimate, abs(I_estimate -
I_true), delta_I))
I_estimate = monte_carlo.mc_importance(n, k=2)
results.append(("Выборка  $p(x) \propto x^2$ ", I_estimate, abs(I_estimate -
I_true), delta_I))
I_estimate = monte_carlo.mc_importance(n, k=3)
results.append(("Выборка  $p(x) \propto x^3$ ", I_estimate, abs(I_estimate -
I_true), delta_I))
I_estimate = monte_carlo.mc_mis(n, use_power_heuristic=False)
results.append(("MIS (средняя плотность)", I_estimate, abs(I_estimate
- I_true), delta_I))
I_estimate = monte_carlo.mc_mis(n, use_power_heuristic=True)
results.append(("MIS (средний квадрат)", I_estimate, abs(I_estimate -
I_true), delta_I))
I_estimate = monte_carlo.mc_russian_roulette(n, R=0.5)
results.append(("Русская рулетка R=0.5", I_estimate, abs(I_estimate -
I_true), delta_I))
I_estimate = monte_carlo.mc_russian_roulette(n, R=0.75)
results.append(("Русская рулетка R=0.75", I_estimate, abs(I_estimate
- I_true), delta_I))
I_estimate = monte_carlo.mc_russian_roulette(n, R=0.95)
results.append(("Русская рулетка R=0.95", I_estimate, abs(I_estimate
- I_true), delta_I))
print_table(results, n)

if __name__ == "__main__":
    main()

```

Выводы и анализ методов

- 1) Простой метод Монте-Карло: дает хорошую точность при большом N.
- 2) Стратификация: улучшает точность, особенно при малом N. Меньший шаг 0.5 дает лучшие результаты, чем шаг 1.0.
- 3) Выборка по значимости:
 - а. $p(x) \propto x$ – дает хорошие результаты, поскольку по форме близка к x^2
 - б. $p(x) \propto x^2$ – особый случай, функция совпадает с исходной, поэтому дисперсия равна нулю

с. $p(x) \propto x^3$ – менее эффективна для данной функции

- 4) Многократная выборка: оба варианта весов дают сравнимую точность, немного улучшая результаты по сравнению с отдельными плотностями.
- 5) Русская рулетка: вносит дополнительную дисперсию. Меньшие значения $R=0.5$ дают более стабильные результаты. При $R=0.95$ возникает наибольшая погрешность.
- 6) Зависимость от N : все методы показывают сходимость к точному значению при увеличении N . Наиболее стабильные результаты при $N \geq 10000$.